

CaptionAI: AI Powered Debt Relief & Financial Recovery Platform

Project Description:

FinRelief AI is an intelligent web-based platform that assists borrowers in managing debt and improving financial recovery through Artificial Intelligence. The system enables users to register securely, manage multiple loans, maintain financial profiles, receive AI-powered settlement predictions, generate personalized negotiation strategies, and automatically create settlement request letters.

The platform uses Machine Learning techniques to analyse borrower financial information—including income, expenses, existing debts, loan details, and repayment history—to estimate settlement possibilities and recommend the optimal settlement amount. It also provides financial health analysis and maintains AI interaction history for future reference.

Built using React.js, Flask, Python, and PostgreSQL, FinRelief AI offers a scalable and secure solution that can be integrated with cloud platforms and future financial service APIs.

Scenarios

Scenario 1

A borrower with multiple outstanding personal loans logs into FinRelief AI. After entering monthly income, expenses, and debt details, the platform calculates a financial health score and recommends the most suitable settlement strategy.

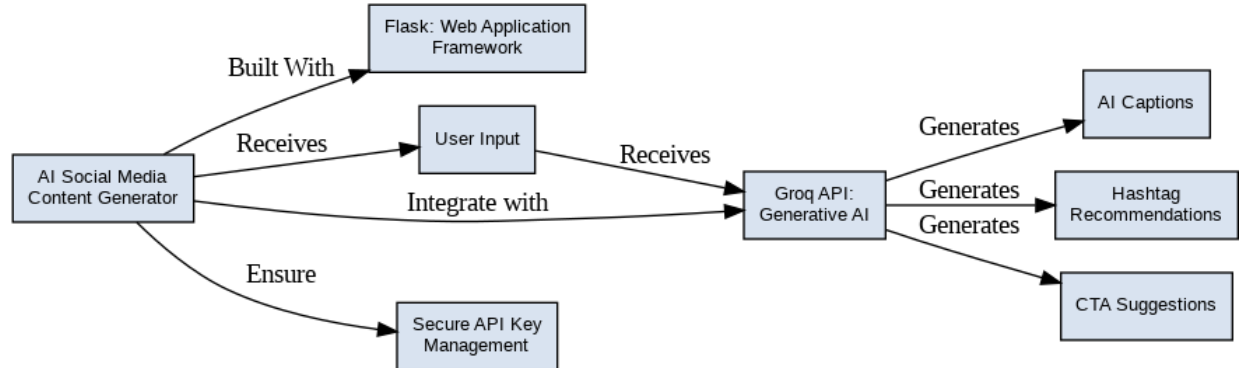
Scenario 2

A borrower uploads loan information for a home loan. The AI predicts an 85% settlement possibility, recommends a settlement amount, and automatically generates a professional negotiation letter to send to the lender.

Scenario 3

A financial advisor accesses borrower records through the admin dashboard to review AI-generated recommendations, monitor financial recovery progress, and assist borrowers in planning repayments.

Technical Architecture:



Description: CaptionAI utilizes a modular three-tier architecture to deliver rapid, AI-driven content generation. The frontend provides a responsive user interface built with HTML and CSS while the Flask-based backend orchestrates data validation and prompt construction. It interfaces with the Groq API using the LLaMA 3.3-70B model to generate tailored social media content, with secure deployment managed locally via Flask and exposed publicly through Ngrok.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

Python 3.11+:

<https://www.python.org/downloads/>

FastAPI Documentation:

<https://fastapi.tiangolo.com/>

React.js Documentation:

<https://react.dev/>

Vite Documentation:

<https://vitejs.dev/>

Google Gemini API:

<https://aistudio.google.com/app/apikey>

SQLAlchemy ORM Documentation:

<https://www.sqlalchemy.org/>

SQLite Documentation:

<https://www.sqlite.org/docs.html>

Git Version Control:

<https://git-scm.com/doc>

GitHub Repository Hosting:

<https://github.com/>

Python Virtual Environment (venv):

<https://docs.python.org/3/library/venv.html>

Visual Studio Code:

<https://code.visualstudio.com/>

JWT Authentication Documentation:

<https://jwt.io/>

Project Workflow:

Epic 1. FinRelief AI Application Development & System Setup

- Story 1: Python Backend Environment Setup
- Story 2: Backend Dependency Installation Using Requirements.txt
- Story 3: Frontend Setup (React.js + Vite)
- Story 4: Project Structure & Full-Stack Directory Organization

Epic 2. AI Integration & Financial Processing Setup

- Story 1: FastAPI Backend & API Endpoints
- Story 2: Financial Engine Module
- Story 3: Settlement Prediction System
- Story 4: AI Negotiation Strategy Engine
- Story 5: Fallback Logic Implementation

Epic 3. Database Management & Financial Data Storage Setup

- Story 1: API Development Functionality
- Story 2: Loan & Settlement Processing Functionality
- Story 3: Data Handling

Epic 4. Frontend Integration & User Interface Development

- Story 1: User Interface Development
- Story 2: Frontend Communication With FastAPI
- Story 3: Financial Health Metrics & Settlement Data Visualization
- Story 4: UI Enhancements

Epic 5. Testing, Debugging & Performance Optimization

- Story 1: System Testing
- Story 2: Backend Error Handling & AI Fallback Management
- Story 3: Performance Optimization & Secure Session Management

Epic 6. Version Control, Project Finalization & Deployment Readiness

- Story 1: GitHub Repository Setup & Version Control Management
- Story 2: Project Cleanup & Modular Folder Structure Organization
- Story 3: Deployment Configuration & Production Readiness Setup

Epic 1: FinRelief AI Application Development & System Setup

It will establish the foundational architecture and development environment of the FinRelief AI platform. It includes configuring the Python backend, React.js frontend, project dependencies, and application structure to support seamless integration of AI-powered financial analysis, authentication, database management, and debt recovery functionalities.

Activity 1.1: Python Backend Environment Setup

The backend of the FinRelief AI – AI-Powered Debt Relief & Financial Recovery Platform is developed using Python 3.10 or above.

A virtual environment is created to isolate dependencies and ensure smooth execution.

Steps performed:

1. Python installed from official website
2. Version verified using: `python --version`
3. Virtual environment created inside project folder: `python -m venv venv`
4. Virtual environment activated: `venv\Scripts\activate`

```
PS C:\Users\Akash\FinReliefAI_> python --version
Python 3.12.9
PS C:\Users\Akash\FinReliefAI_>
PS C:\Users\Akash\FinReliefAI_> python -m venv venv
PS C:\Users\Akash\FinReliefAI_> venv\Scripts\activate
(venv) PS C:\Users\Akash\FinReliefAI_> |
```

Activity 1.2: Backend Dependency Installation All backend dependencies are installed using **requirements.txt**.

Key dependencies include:

1. fastapi
2. uvicorn
3. sqlalchemy
4. python-jose
5. werkzeug
6. python-dotenv
7. psycpg2-binary
8. google-generativeai
9. Installation command: `pip install -r requirements.txt`

```

PS C:\Users\Akash\FinReliefAI_\app> cd ..
PS C:\Users\Akash\FinReliefAI_> pip install -r requirements.txt
WARNING: Ignoring invalid distribution ~vicorn (C:\Users\Akash\miniconda3\Li
b\site-packages)
Collecting fastapi==0.109.0 (from -r requirements.txt (line 1))
  Using cached fastapi-0.109.0-py3-none-any.whl.metadata (24 kB)
Collecting uvicorn==0.27.0 (from -r requirements.txt (line 2))
  Using cached uvicorn-0.27.0-py3-none-any.whl.metadata (6.4 kB)
Collecting sqlalchemy==2.0.25 (from -r requirements.txt (line 3))
  Downloading SQLAlchemy-2.0.25-cp312-cp312-win_amd64.whl.metadata (9.8 kB)
Collecting pydantic==2.5.3 (from -r requirements.txt (line 4))
  Using cached pydantic-2.5.3-py3-none-any.whl.metadata (65 kB)
Requirement already satisfied: python-jose==3.3.0 in c:\users\akash\minicond
a3\lib\site-packages (from python-jose[cryptography]==3.3.0->-r requirements
.txt (line 6)) (3.3.0)
Requirement already satisfied: passlib==1.7.4 in c:\users\akash\miniconda3\l
ib\site-packages (from passlib[bcrypt]==1.7.4->-r requirements.txt (line 7))
(1.7.4)
Collecting python-multipart==0.0.6 (from -r requirements.txt (line 8))
  Downloading python_multipart-0.0.6-py3-none-any.whl.metadata (2.5 kB)
Collecting requests==2.31.0 (from -r requirements.txt (line 9))
  Using cached requests-2.31.0-py3-none-any.whl.metadata (4.6 kB)
Collecting python-dotenv==1.0.0 (from -r requirements.txt (line 10))
  Using cached python_dotenv-1.0.0-py3-none-any.whl.metadata (21 kB)
Requirement already satisfied: pycopp2-binary==2.9.9 in c:\users\akash\mini
conda3\lib\site-packages (from -r requirements.txt (line 11)) (2.9.9)
Collecting starlette<0.36.0,>=0.35.0 (from fastapi==0.109.0->-r requirements
.txt (line 1))
  Using cached starlette-0.35.1-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\akash\mi
niconda3\lib\site-packages (from fastapi==0.109.0->-r requirements.txt (line
1)) (4.15.0)
Requirement already satisfied: click>=7.0 in c:\users\akash\miniconda3\lib\s
ite-packages (from uvicorn==0.27.0->-r requirements.txt (line 2)) (8.3.1)
Requirement already satisfied: h11>=0.8 in c:\users\akash\miniconda3\lib\sit

```

Activity 1.3: Frontend Setup (React / Vite) The frontend is built using React.js and Vite to provide a fast and responsive user interface.

Steps:

1. npm install

```

PS C:\Users\Akash> cd FinReliefAI_
PS C:\Users\Akash\FinReliefAI_> cd C:\Users\Akash\FinReliefAI_\frontend
PS C:\Users\Akash\FinReliefAI_\frontend> npm install

added 186 packages, and audited 187 packages in 20s

40 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

2. npm run dm

```
PS C:\Users\Akash\FinReliefAI_\frontend> npm run dev

> frontend@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

VITE v7.3.3  ready in 530 ms

→ Local:   http://localhost:5174/
→ Network: use --host to expose
→ press h + enter to show help
12:27:27 pm [vite] (client) hmr update /src/pages/Login.jsx
12:27:27 pm [vite] (client) hmr update /src/pages/Login.jsx (x2)
12:29:59 pm [vite] (client) hmr update /src/pages/Login.jsx (x3)
```

Activity 1.4: Project Structure Setup

Project Structure Setup The project follows a modular full-stack architecture with separate modules for frontend pages, backend APIs, authentication, financial engine, settlement engine, and AI negotiation strategy generation.

```
(venv) PS C:\Users\Akash\FinReliefAI_> dir

Directory: C:\Users\Akash\FinReliefAI_

Mode                LastWriteTime         Length Name
----                -
d-----          10-05-2026         11:46      app
d-----          10-05-2026         11:47      frontend
d-----          15-05-2026         12:34      venv
d-----          10-05-2026         11:39      {app
-a-----          11-05-2026          11:53          131 .env
-a-----          11-05-2026          11:41          200 .gitignore
-a-----          11-05-2026          11:31        40960 finrelief.db
-a-----          10-05-2026          11:39        4235 README.md
-a-----          10-05-2026          11:39          355 requirements.txt

(venv) PS C:\Users\Akash\FinReliefAI_> |
```

It will integrate Artificial Intelligence and financial analysis capabilities into the platform. It includes implementing financial calculation modules, settlement prediction mechanisms, AI-powered negotiation strategy generation, backend API development, and intelligent decision-making features using Google Gemini AI.

Activity 2.1: FastAPI Backend & API Endpoints

FastAPI Backend & API Endpoints The system implements a clean REST API using FastAPI with multiple endpoints handling user authentication, loan management, financial health calculation, settlement prediction, and AI negotiation strategy generation.

FinRelief AI



0.1.0

OAS 3.1

/openapi.json

Authorize



default

POST /register Register

GET /debug-user Debug User

POST /login Login

PUT /update-profile Update Profile

POST /add-loan Add Loan

GET /loans Get Loans

DELETE /delete-loan/{loan_id} Delete Loan

GET /dashboard-data Get Dashboard Data

GET /settlement-predictor Get Settlement Prediction

GET /ai-negotiation-strategy Get Ai Negotiation Strategy

GET /generate-negotiation-email/{loan_id} Generate Negotiation Email

GET /ai-history Get Ai History

GET /financial-health Get Financial Health

GET /debt-timeline Get Debt Timeline

```
PS C:\Users\Akash\FinReliefAI_> uvicorn app.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\Akash\\F
inReliefAI_']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [23092] using WatchFiles
INFO: Started server process [26748]
INFO: Waiting for application startup.
✅ Database Tables Created Successfully
INFO: Application startup complete.
WARNING: WatchFiles detected changes in 'frontend\\node_modules\\flatted\\pyth
on\\flatted.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [26748]
INFO: Started server process [26964]
INFO: Waiting for application startup.
✅ Database Tables Created Successfully
INFO: Application startup complete.
INFO: 127.0.0.1:64879 - "OPTIONS /register HTTP/1.1" 200 OK
INFO: 127.0.0.1:64879 - "POST /register HTTP/1.1" 200 OK
```

```
# Root Route

@app.get("/")
def read_root():
    return {"message": "Welcome to FinRelief AI 🚀", "status": "running"}

# Test Database Connection

@app.get("/test-db")
def test_db():
    with engine.connect() as connection:
        connection.execute(text("SELECT 1"))
    return {"database_status": "Connected ✅"}
```

Activity 2.2 : Financial Engine Module

Financial Engine Module The platform calculates real-time financial health metrics including EMI-to-income ratio, debt-to-income ratio, monthly surplus, total outstanding debt, and financial stress level based on the borrower's income, expenses, and loan portfolio.

Function: EMI & Debt-to-Income Ratio Calculation.

Calculates EMI ratio and debt-to-income percentage using borrower monthly income, expenses, and total loan obligations for financial health analysis

```
def calculate_financial_health(user, loans):
    """Calculate financial health metrics"""
    total_emi = sum(loan.emi for loan in loans)
    total_outstanding = sum(loan.outstanding_amount for loan in loans)
    surplus = user.monthly_income - user.monthly_expenses - total_emi

    # EMI Ratio = (total_emi / monthly_income) * 100
    if user.monthly_income > 0:
        emi_ratio = (total_emi / user.monthly_income) * 100
    else:
        emi_ratio = 0

    # Debt-to-Income = (total_outstanding / monthly_income) * 100
    if user.monthly_income > 0:
        debt_to_income = (total_outstanding / user.monthly_income) * 100
    else:
        debt_to_income = 0
```

Function: Financial Stress Level Classification

Classifies borrower debt stress into Low, Medium, and High categories based on calculated EMI ratio and repayment burden analysis.

```
def calculate_financial_health(user, loans):

    # Stress Level Logic:
    # <30 → Low
    # 30-50 → Medium
    # >50 → High
    if emi_ratio > 50:
        stress_level = "High"
    elif emi_ratio >= 30:
        stress_level = "Medium"
    else:
        stress_level = "Low"

    return {
        "total_emi": total_emi,
        "total_outstanding": total_outstanding,
        "surplus": surplus,
        "emi_ratio_percent": round(emi_ratio, 2),
        "debt_to_income_percent": round(debt_to_income, 2),
        "stress_level": stress_level,
        "total_loans": len(loans)
    }
```

Function: Loan Priority Analysis

Analyzes borrower loan portfolio and assigns priority levels based on overdue duration, interest rate, and EMI burden for intelligent debt management.

```
# Sort by priority (High first)
priority_order = {"High": 0, "Medium": 1, "Low": 2}
priority_list.sort(key=lambda x: priority_order[x["priority"]])

return priority_list

def simulate_debt_timeline(user, loans, extra_payment=0):
    loan_data = [
        {
            "loan_id": loan.id,
            "balance": loan.outstanding_amount,
            "interest_rate": loan.interest_rate,
            "emi": loan.emi
        }
        for loan in loans
    ]

    months = 0
    max_months = 240
    timeline = []

    while any(loan["balance"] > 0 for loan in loan_data) and months < max_months:
        months += 1
        total_balance = 0

        loan_data.sort(key=lambda x: x["balance"], reverse=True)
```

Function: Priority-Based Loan Sorting

Sorts loan records according to financial urgency and repayment priority to improve settlement planning and negotiation strategy generation.

```
for loan in loan_data:
    if loan["balance"] <= 0:
        continue

    monthly_interest = (loan["interest_rate"] / 100) / 12
    loan["balance"] += loan["balance"] * monthly_interest

    payment = loan["emi"]

    if extra_payment > 0 and loan == loan_data[0]:
        payment += extra_payment

    loan["balance"] -= payment

    if loan["balance"] < 0:
        loan["balance"] = 0

    total_balance += loan["balance"]

    timeline.append({
        "month": months,
        "remaining_balance": round(total_balance, 2)
    })
```

Function: Debt Repayment Timeline Simulation

Simulates monthly debt repayment progression and balance reduction scenarios based on borrower repayment capacity and additional payment strategies.

```
    })

    return {
        "months_to_debt_free": months,
        "final_remaining_balance": round(total_balance, 2),
        "timeline_preview": timeline[:12]
    }
```

Activity 2.3 : Settlement Prediction System

The system calculates optimal settlement percentages and risk categories for each loan by analyzing outstanding amount, overdue months, loan type, and borrower financial profile to provide data-driven settlement recommendations.

```
# Calculate Debt-to-Income ratio
if user.monthly_income > 0:
    debt_to_income = (total_outstanding / user.monthly_income) * 100
else:
    debt_to_income = 0

settlement_results = []

for loan in loans:
    # Base settlement = 50%
    settlement = 50.0
    risk_score = 0

    # +5% if overdue
    if loan.overdue_months > 0:
        settlement += 5
        risk_score += 20

    # +5% if EMI ratio > 50
    if emi_ratio > 50:
        settlement += 5
        risk_score += 15
```

```
# +5% if interest_rate > 12%
if loan.interest_rate > 12:
    settlement += 5
    risk_score += 10

# +5% if debt_to_income > 80%
if debt_to_income > 80:
    settlement += 5
    risk_score += 15

# Clamp settlement: Minimum 40%, Maximum 75%
settlement = max(40, min(75, settlement))

# Determine risk category
if risk_score >= 40:
    risk_category = "High"
elif risk_score >= 20:
    risk_category = "Medium"
else:
    risk_category = "Low"

settlement_results.append({
    "loan_id": loan.id,
    "lender_name": loan.lender_name,
    "outstanding_amount": loan.outstanding_amount,
    "interest_rate": loan.interest_rate,
    "emi": loan.emi,
```

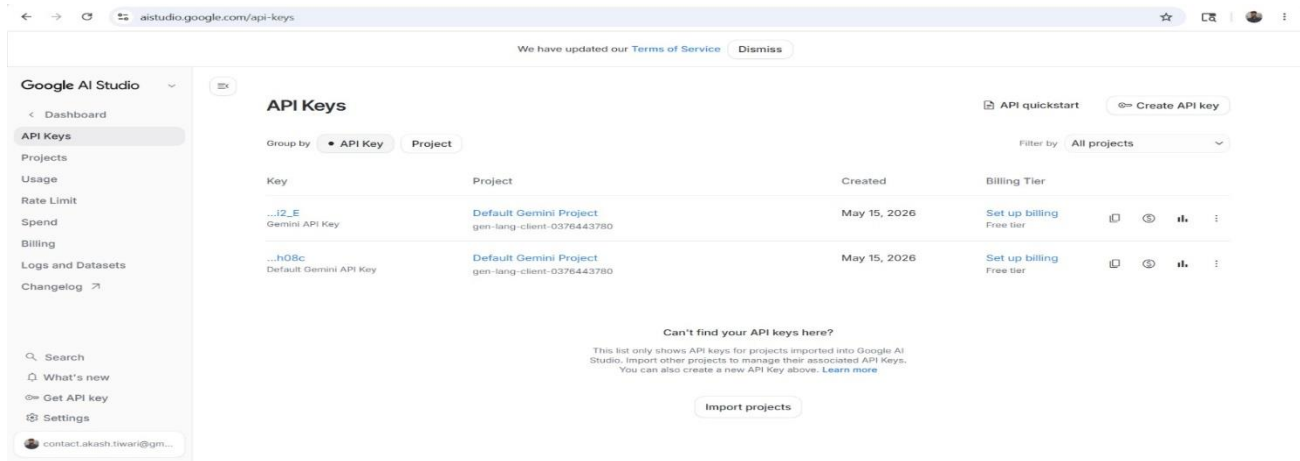
```
        "suggested_settlement_percentage": settlement,
        "risk_score": risk_score,
        "risk_category": risk_category
    })

return settlement_results
```

Activity 2.4 : AI Negotiation Strategy Engine

An AI-powered negotiation strategy system is implemented to:

1. Analyze borrower's complete financial profile and loan data
2. Generate personalized settlement strategies using Google Gemini API
3. Produce lender-specific professional negotiation letters
4. Activate rule-based fallback engine when API key is unavailable



```
import os
import json
from dotenv import load_dotenv

load_dotenv()

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY", "")

def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        import google.generativeai as genai
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Activity 2.5 : Fallback Logic Implementation

Rule-based fallback mechanisms and error handling modules are implemented to ensure system stability and uninterrupted AI functionality without an active Gemini API key.

```
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY", "")

def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        genai = importlib.import_module("google.generativeai")
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Epic 3: Database Management & Financial Data Storage Setup

It will configure and manages the database infrastructure required for secure storage and processing of borrower information, loan records, settlement data, and AI-generated negotiation history. It ensures efficient data handling, structured relationships, authentication support, and reliable communication between system components.

Activity 3.1 : API Development Functionality:

1. Accepts frontend requests with loan details and financial profile
2. Validates input using Pydantic schemas
3. Calls Google Gemini API via ai_engine.py
4. Returns structured JSON responses to frontend

```
@router.get("/ai-negotiation-strategy")
def get_ai_negotiation_strategy(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        user = db.query(User).filter(User.id == current_user.id).first()
        loans = db.query(Loan).filter(Loan.user_id == user.id).all()
        if not loans:
            return {"strategy": "Please add at least one loan to generate an AI strategy."}
        financial_health = calculate_financial_health(user, loans)
        settlement_data = calculate_settlement_probability(user, loans)
        strategy = generate_negotiation_strategy(user, loans, financial_health, settlement_data)
        try:
            db.add(AIHistory(user_id=user.id, query_type="Negotiation Strategy", response=strategy))
            db.commit()
        except Exception:
            db.rollback()
        return {"strategy": strategy}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"AI Strategy error: {str(e)}")
```

Activity 3.2 : Loan & Settlement Processing Functionality

1. Stores and retrieves loan records using SQLAlchemy ORM
2. Calculates settlement percentage based on overdue months and loan type
3. Assigns High, Medium, or Low priority to each loan
4. Generates negotiation letters with lender-specific content

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./finrelief.db")

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

Activity 3.3: Data Handling

1. Structured loan and user data stored in SQLite database
2. Efficient query retrieval per user session
3. JWT token-based authentication for all protected routes

```
from sqlalchemy import Column, Integer, Float, String
from app.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)

    # 🛡️ Authentication Fields
    email = Column(String(255), unique=True, index=True, nullable=False)
    password = Column(String(255), nullable=False)

    # 💰 Financial Profile
    monthly_income = Column(Float, nullable=False, default=0.0)
    monthly_expenses = Column(Float, nullable=False, default=0.0)
    lump_sum_available = Column(Float, nullable=False, default=0.0)
```

Epic 4: Frontend Integration & User Interface Development

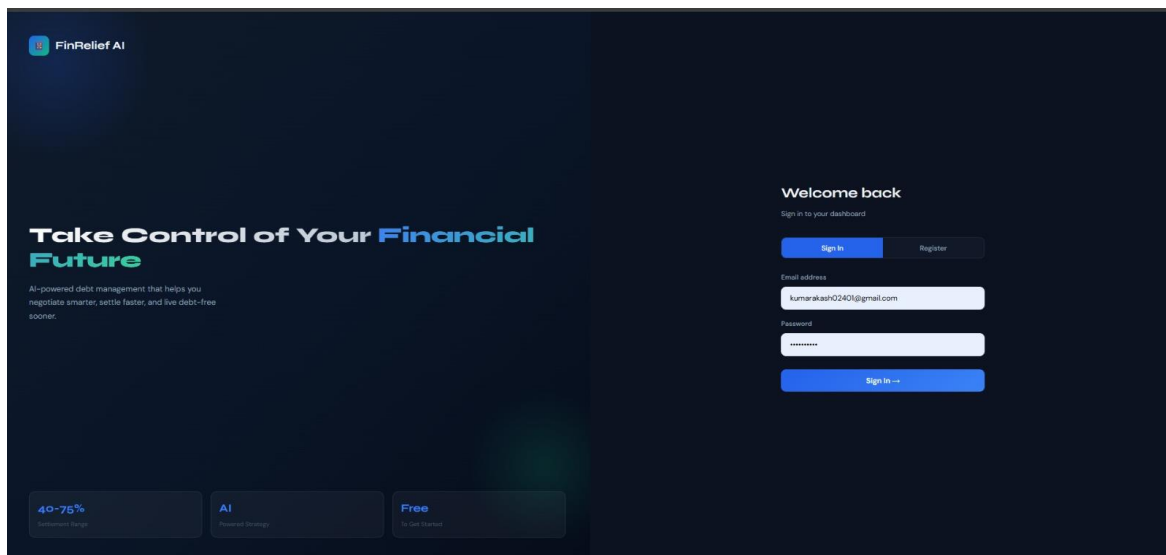
In this we will develop and integrate the user-facing components of the FinRelief AI platform using React.js and Vite. It provides responsive dashboards, financial data visualizations, settlement insights, and seamless interaction between users and backend services through secure API communication.

Activity 4.1: User Interface Development

User Interface Development The frontend is built using React.js and Vite providing a modern dark-themed professional interface.

Users can:

1. Register and login securely on the Login page
2. View financial overview on the Dashboard page
3. View settlement predictions on the Settlement Predictor page
4. Generate negotiation letters on the Negotiation Email page
5. View borrower rights on the Know Your Rights page
6. Review past AI outputs on the History page



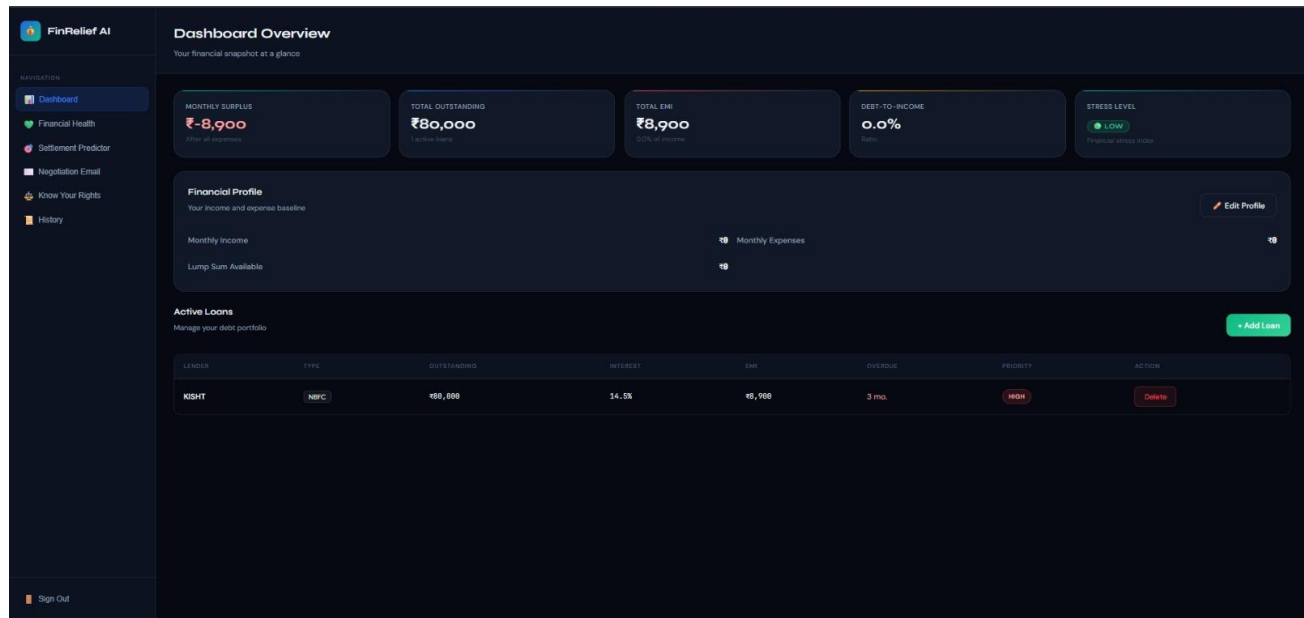
Activity 4.2 : Frontend communicates with FastAPI

Backend Integration Frontend communicates with FastAPI backend APIs through Axios with JWT interceptors to process requests and display real-time financial data and AI-generated negotiation content.

Key Functionalities:

1. Frontend communicates with FastAPI backend APIs using Axios
2. JWT interceptors securely attach authentication tokens to API requests
3. Real-time financial data is displayed dynamically on the dashboard
4. Backend processes borrower loan and financial profile data

5. Settlement prediction results are fetched from backend APIs
6. AI-generated negotiation strategies are displayed instantly
7. Axios interceptors handle unauthorized responses automatically
8. Secure authentication flow is maintained across protected routes
9. Smooth communication between frontend and backend modules ensured
10. Responsive dashboard visualization improves borrower interaction experience



Activity 4.3: Financial Health Metrics & Settlement Data Visualization

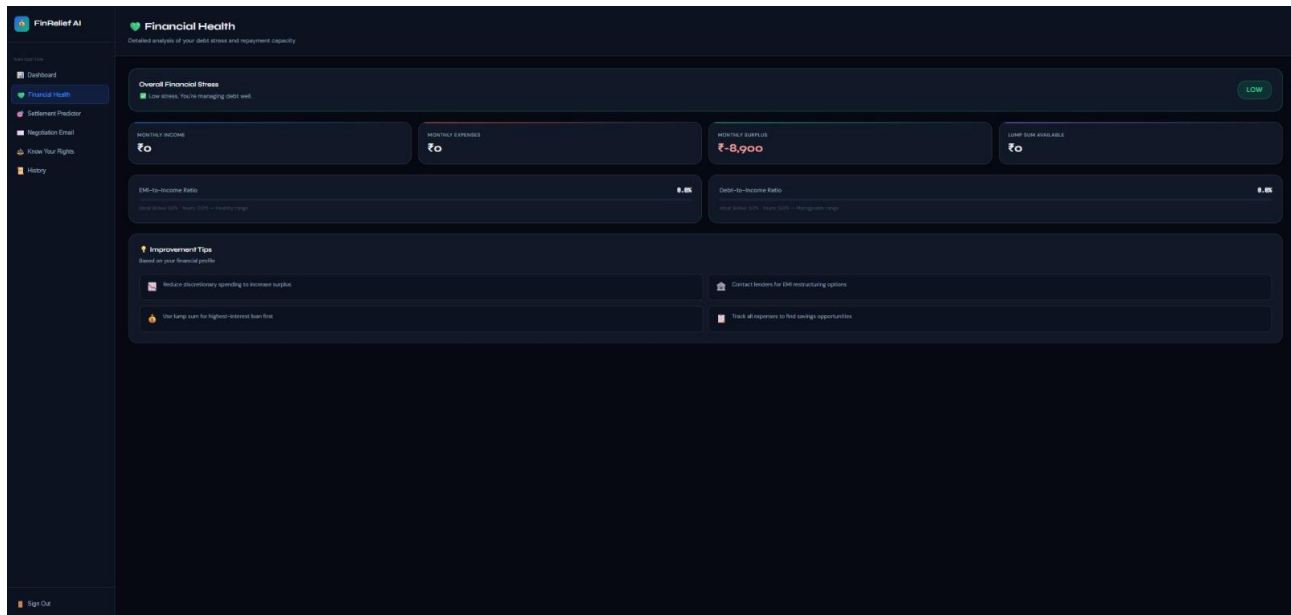
The Financial Data Visualization module provides borrowers with a clear and interactive overview of their financial condition through dynamic dashboard components and real-time analytics. The system visually represents important financial metrics such as monthly surplus, EMI-to-income ratio, debt stress level, settlement percentages, and overall financial health status to help users better understand their debt recovery progress and financial stability.

Key Functionalities:

1. Monthly surplus metrics displayed dynamically on the dashboard
2. EMI-to-income ratio visualization for financial analysis
3. Debt stress level badges generated based on borrower financial profile
4. Settlement percentage displayed for each loan account
5. Financial health progress meters integrated for borrower monitoring
6. Real-time financial analytics fetched through backend APIs
7. Interactive dashboard cards for financial overview visualization
8. Responsive UI components for better user experience
9. Color-coded indicators for risk and stress level analysis
10. Smooth frontend rendering of borrower financial insights

11. Financial Dashboard Interface

12. Displays borrower financial overview including active loans, monthly surplus, debt stress analysis, and real-time financial health metrics through an interactive dashboard.



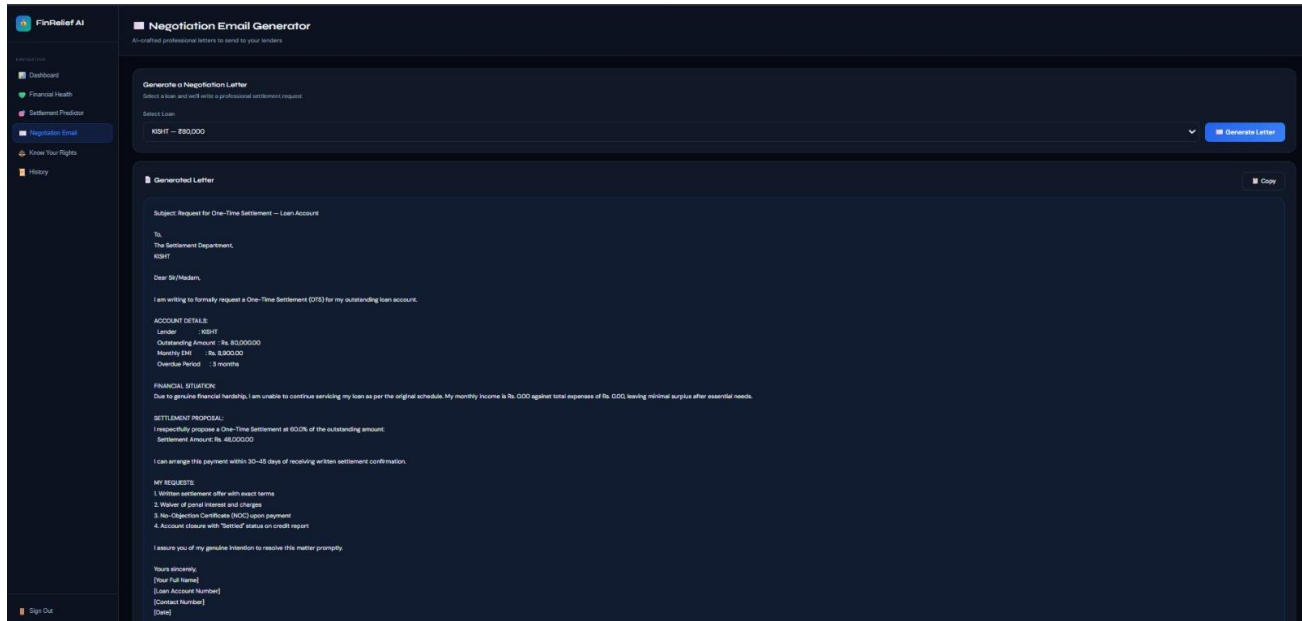
AI-Powered Settlement Prediction

Generates intelligent settlement recommendations and repayment analysis based on borrower financial profile and overdue loan information.



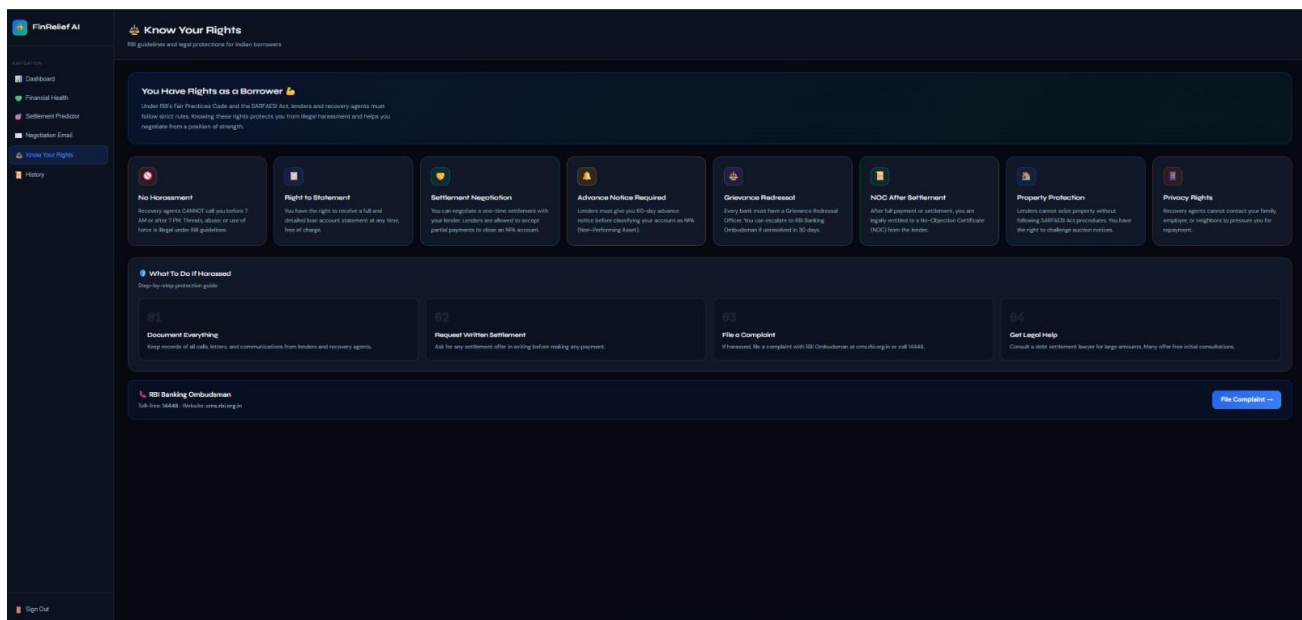
AI Negotiation Strategy Generation

Creates lender-specific AI-powered negotiation letters and settlement request content using Google Gemini API integration.



Borrower Rights & Financial Guidance Module

Provides financial awareness resources, borrower rights information, and AI-assisted debt management guidance through an organized user interface.



Activity 4.4 : UI Enhancements

The UI Enhancement module focuses on improving the visual appearance, responsiveness, accessibility, and overall user experience of the FinRelief AI platform. A modern dark-themed professional interface is developed using custom CSS variables, responsive layouts, interactive components, and smooth frontend transitions to create a clean and borrower-friendly financial dashboard environment.

Key Functionalities:

1. Dark professional layout implemented using CSS variables
2. Responsive design optimized for desktop and mobile devices
3. Custom component styling for dashboard and financial modules
4. Smooth modal transitions and frontend animations integrated
5. Interactive form handling for better user experience
6. Borrower-friendly clean and modern interface developed
7. Consistent UI theme maintained across all application pages
8. Modern dashboard cards and navigation panels implemented
9. Enhanced readability and accessibility through optimized styling
10. Responsive frontend components improve borrower interaction and usability

```
:root {  
  --bg-base: #060912;  
  --bg-surface: #0d1220;  
  --bg-card: #111827;  
  --bg-hover: #1a2235;  
  --border: rgba(255,255,255,0.08);  
  --border-bright: rgba(255,255,255,0.15);  
  
  --accent-blue: #2563eb;  
  --accent-blue-light: #3b82f6;  
  --accent-green: #10b981;  
  --accent-green-light: #34d399;  
  --accent-red: #ef4444;  
  --accent-amber: #f59e0b;  
  --accent-purple: #8b5cf6;  
  
  --text-primary: #f1f5f9;  
  --text-secondary: #94a3b8;  
  --text-muted: #475569;  
  
  --font-display: 'Syne', sans-serif;  
  --font-body: 'DM Sans', sans-serif;  
  --font-mono: 'JetBrains Mono', monospace;  
  
  --radius: 12px;  
  --radius-lg: 20px;  
  --radius-xl: 28px;  
  --sidebar-width: 260px;
```

Epic 5: Testing, Debugging & Performance Optimization

It validates, tests, and optimizes the overall functionality of the platform. It includes system testing, error handling, AI fallback management, performance improvements, secure session handling, and refinement of borrower-facing features to ensure reliable and stable application performance.

Activity 5.1 : System Testing

The system testing phase focuses on validating the stability, accuracy, and reliability of the FinRelief AI platform across multiple borrower financial scenarios and loan management workflows. Different loan types, financial profiles, settlement prediction cases, and AI-generated negotiation processes are tested to ensure smooth platform functionality and accurate financial analysis under real-world conditions.

Key Functionalities:

1. Multiple loan types tested for financial processing accuracy
2. Different borrower financial profile scenarios validated
3. Settlement prediction calculations verified across various debt conditions
4. Missing Google Gemini API key scenarios tested successfully
5. Invalid user input handling and validation implemented
6. Frontend and backend integration testing performed
7. API response validation conducted for financial modules
8. AI-generated negotiation strategy outputs tested
9. Error scenarios and edge cases handled efficiently
10. Stable system behavior verified under different financial conditions

```
@router.put("/update-profile")
def update_profile(profile: ProfileUpdate, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        user = db.query(User).filter(User.id == current_user.id).first()
        user.monthly_income = profile.monthly_income
        user.monthly_expenses = profile.monthly_expenses
        user.lump_sum_available = profile.lump_sum_available
        db.commit()
        return {"message": "Profile Updated Successfully"}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Failed to update profile: {str(e)}")
```

```
@router.post("/add-loan")
def add_loan(loan: LoanCreate, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    try:
        new_loan = Loan(
            user_id=current_user.id, lender_name=loan.lender_name,
            outstanding_amount=loan.outstanding_amount, interest_rate=loan.interest_rate,
            emi=loan.emi, overdue_months=loan.overdue_months, loan_type=loan.loan_type
        )
        db.add(new_loan)
        db.commit()
        db.refresh(new_loan)
        return {"message": "Loan Added Successfully", "id": new_loan.id}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Failed to add loan: {str(e)}")
```

Activity 5.2 : Backend Error Handling & AI Fallback Management

The Error Handling module is designed to ensure stable, secure, and uninterrupted operation of the FinRelief AI platform during financial processing and AI-powered negotiation workflows. Structured exception handling mechanisms and rule-based fallback systems are

implemented across backend modules to manage API failures, invalid user inputs, unexpected server responses, and AI generation issues efficiently.

Key Functionalities:

1. try-except blocks implemented across backend processing modules
2. Rule-based fallback system activated during Gemini API failures
3. Stable AI-generated responses ensured without active API access
4. Invalid user input handling and validation implemented
5. Frontend displays user-friendly error and validation messages
6. API timeout and unexpected response handling configured
7. Backend exception management prevents application crashes
8. Structured error responses returned through FastAPI APIs
9. Smooth frontend-backend communication maintained during failures
10. Improved platform stability and reliable borrower experience

```
def _call_gemini(prompt: str) -> str:
    """Call Google Gemini API if key is available, otherwise use rule-based fallback."""
    if not GOOGLE_API_KEY:
        return None # Will fall through to fallback

    try:
        genai = importlib.import_module("google.generativeai")
        genai.configure(api_key=GOOGLE_API_KEY)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(prompt)
        return response.text
    except ImportError:
        return None
    except Exception as e:
        print(f"Gemini API error: {e}")
        return None
```

Activity 5.3 : Performance Optimization & Secure Session Management

The performance optimization phase focuses on improving the efficiency, stability, authentication handling, and overall responsiveness of the FinRelief AI platform. Backend processing, database operations, authentication workflows, and API communication mechanisms are optimized to ensure smooth financial processing, fast response handling, and reliable borrower interaction across all application modules.

Key Functionalities:

1. JWT token expiry configured for 120-minute secure session management
2. SQLite optimized with check_same_thread=False for concurrent request handling
3. Axios interceptors implemented to prevent duplicate authentication headers
4. Werkzeug password hashing optimized for secure and fast verification
5. Backend API response handling improved for better performance
6. Database query execution optimized for real-time financial processing
7. Authentication workflow enhanced for stable user session management
8. Frontend-backend communication latency reduced
9. Efficient handling of concurrent borrower requests implemented
10. Overall application responsiveness and scalability improved

```
@router.post("/login")
def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db),
):
    try:
        # OAuth2PasswordRequestForm uses "username" field to carry whatever the client sends.
        email = (form_data.username or "").strip().lower()
        password = form_data.password or ""

        user = db.query(User).filter(User.email == email).first()
        if not user:
            logger.warning("Login failed: user not found for email=%s", email)
            raise HTTPException(status_code=401, detail="Invalid email or password")

        if not verify_password(password, user.password):
            logger.warning("Login failed: password mismatch for email=%s", email)
            raise HTTPException(status_code=401, detail="Invalid email or password")

        token = create_token({"sub": user.email})
        return {"access_token": token, "token_type": "bearer"}
    except HTTPException:
        raise
    except Exception as e:
        logger.exception("Login failed with server error")
        raise HTTPException(status_code=500, detail=f"Login failed: {str(e)}")
```

Epic 6: Version Control, Project Finalization & Deployment Readiness

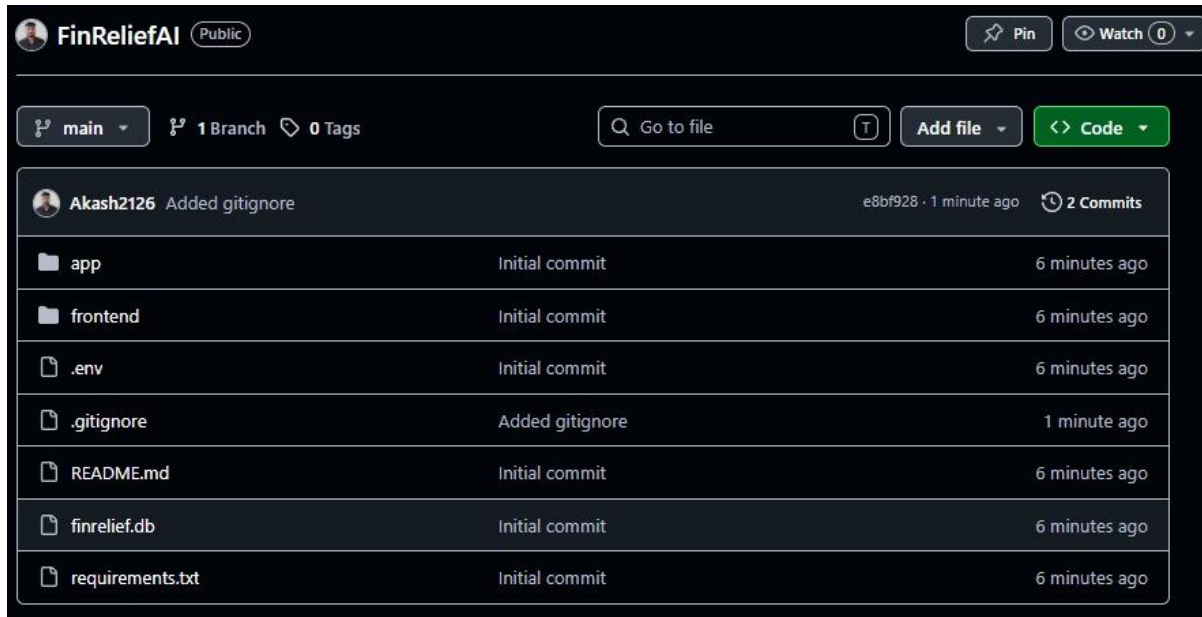
It will prepare the FinRelief AI platform for deployment and production use. It includes project cleanup, modular architecture organization, GitHub version control management, deployment configuration, and environment setup to ensure secure, scalable, and maintainable application delivery.

Activity 6.1: GitHub Repository Setup & Version Control Management

The GitHub Repository Setup phase focuses on maintaining organized source code management, version tracking, and collaborative development workflows for the FinRelief AI platform. The complete project source code, including frontend, backend, database modules, and AI integration components, is pushed to GitHub to ensure secure repository management and streamlined development practices.

Key Functionalities:

1. Project source code pushed successfully to GitHub repository
2. Git-based version control workflow implemented
3. Backend and frontend modules organized systematically
4. Repository structure maintained for scalable development
5. Commit history tracked for project modifications and updates
6. Secure source code backup and collaboration enabled
7. Branch management and code version tracking implemented
8. Project files synchronized with GitHub repository
9. Development workflow streamlined through Git integration
10. Version control maintained for future deployment and enhancements



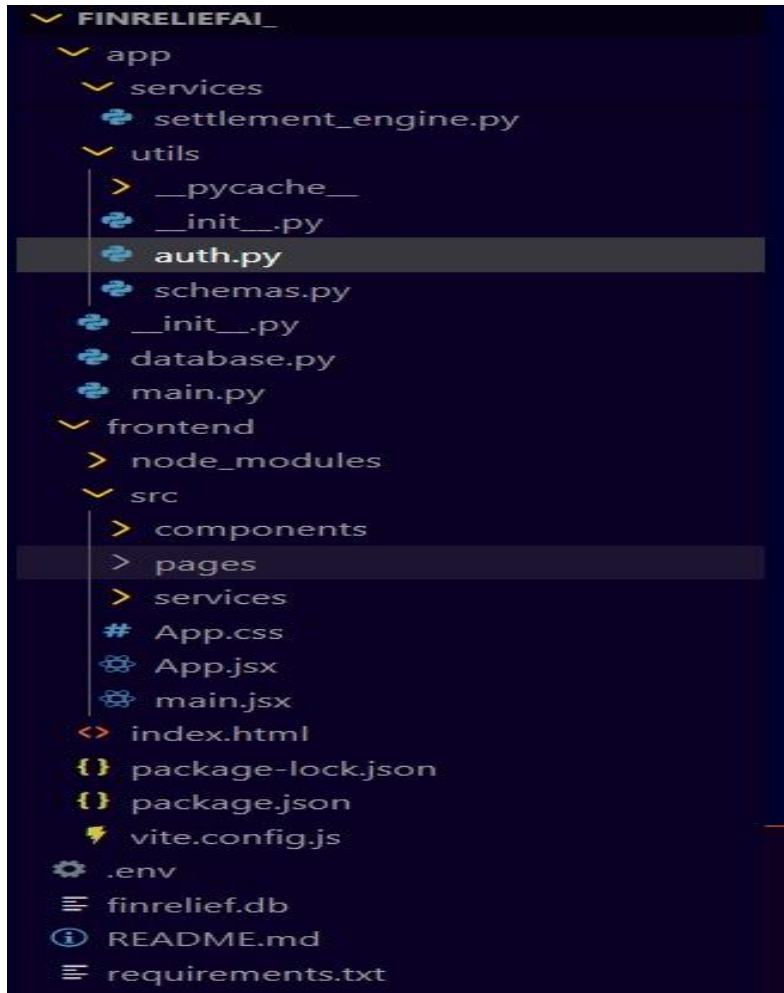
Activity 6.2: Project Cleanup & Modular Folder Structure Organization

The project cleanup phase focuses on organizing the FinRelief AI application into a clean, scalable, and maintainable modular architecture. Unnecessary temporary files, cache directories, and unused resources are removed to optimize project performance and improve development workflow efficiency.

The application structure is reorganized into separate modules for app configuration, API routes, database models, business services, frontend components, and utility functions to ensure better code readability, maintainability, and deployment readiness.

Key Functionalities:

1. Unnecessary files and cache directories removed successfully
2. Project structure organized into modular folders and components
3. Separate app, routes, models, services, and utils modules maintained
4. Backend and frontend source code cleaned and optimized
5. Improved code readability and maintainability achieved
6. Scalable folder architecture implemented for future enhancements
7. Duplicate and unused resources removed from the project
8. Modular organization improved development workflow efficiency
9. Structured source code management prepared for deployment
10. Clean and professional project architecture maintained



Activity 6.3: Deployment Configuration & Production Readiness Setup

The deployment readiness phase focuses on preparing the FinRelief AI platform for production deployment by configuring frontend builds, backend server setup, and environment variable management. The application is optimized to ensure stable execution, scalable deployment, secure API communication, and smooth integration of all financial processing and AI-powered modules in a production environment.

The frontend application is prepared using Vite production build optimization, while backend APIs are configured using the Uvicorn ASGI server for high-performance request handling and scalable FastAPI deployment. Environment variables are securely managed through .env configuration files to maintain safe handling of database credentials, API keys, authentication secrets, and deployment settings.

Key Functionalities:

- Frontend prepared for deployment using Vite production build
- Backend APIs configured with Uvicorn ASGI server
- Environment variables successfully configured in .env file
- Production-ready FastAPI backend deployment setup implemented
- Secure handling of API keys and authentication secrets ensured
- Frontend and backend deployment workflow validated
- Optimized production build generated for scalable application hosting
- Deployment configuration organized for cloud/server environments
- API communication and authentication flow verified for production mode

j. Application prepared for stable and efficient deployment execution

```
PS C:\Users\Akash\FinReliefAI_> cd frontend
PS C:\Users\Akash\FinReliefAI_\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v7.3.3 building client environment for production...
✓ 102 modules transformed.
dist/index.html          0.46 kB | gzip: 0.29 kB
dist/assets/index-BtTpTUmN.css  20.42 kB | gzip: 4.07 kB
dist/assets/index-B9DXnwFD.js 312.10 kB | gzip: 99.19 kB
✓ built in 1.30s
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.14.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.14.1
npm notice To update run: npm install -g npm@11.14.1
npm notice
PS C:\Users\Akash\FinReliefAI_\frontend> |
```

Epic 7 : Conclusion

The **FinRelief AI – AI-Powered Debt Relief & Financial Recovery Platform** demonstrates how AI and modern full-stack technologies can simplify debt management and financial recovery. Built with **React.js**, **FastAPI**, **Python**, and **SQLite**, the platform provides intelligent features such as financial health analysis, settlement prediction, AI-generated negotiation strategies, and secure user authentication. Its modular architecture ensures scalability, reliability, and efficient performance. Overall, the system reduces manual effort, empowers borrowers with personalized financial insights, and offers an effective AI-driven solution for modern debt relief and recovery.

